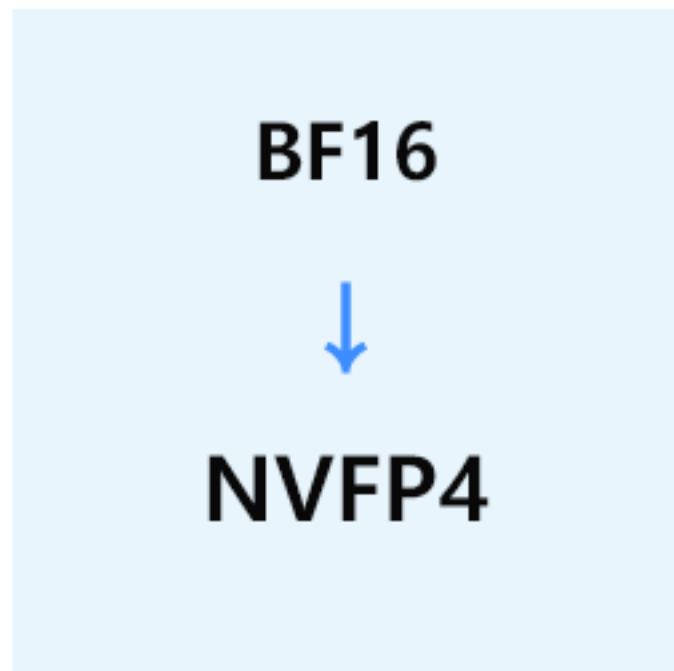


4비트로 내려가도 능력을 지킬 수 있을까?



Quantization 기초 · Native NVFP4 Training · QAD · NVIDIA production

양자화 원리 · NVFP4 · PTQ/QAT/QAD · NVIDIA production stack

이 발표의 한 문장

**FP4의 핵심은 '4비트'가 아니라
오차를 어디서, 어떻게 통제하느냐입니다.**

Native training은 수렴을 지키고, QAD는 완성 모델의 행동을 복구합니다.

두 논문은 서로 다른 문제를 푼다

	문제	오늘 답할 질문
1	Quantization 기초	숫자를 4bit로 줄이면 어떤 오차가 생길까?
2	Native NVFP4 training	처음부터 FP4 GEMM으로 학습해도 어떻게 수렴할까?
3	QAD accuracy recovery	완성 BF16 모델을 NVFP4로 바꾼 뒤 행동을 어떻게 복구할까?
4	Production / Q&A	실제 NVIDIA stack에서는 무엇을 섞고, 무엇을 아직 못 푸나?

양자화의 이득은 메모리·대역폭·GEMM에서 온다

1B parameters의 weight payload

BF16

16 bit/value

FP8

8 bit/value

NVFP4

4.5 bit/value

왜 빨라질까?

- 더 적은 HBM 읽기
- 더 많은 값이 cache에 적재
- Blackwell Tensor Core의 native FP4 GEMM

주의: optimizer·KV cache·scale metadata까지 모두 4비트라는 뜻은 아닙니다.

비트 수가 같아도 숫자 체계는 다르다

형식	비트	구조	강점	대표 약점
FP32	32	E8M23	넓은 범위·높은 정밀도	메모리·연산량
BF16	16	E8M7	FP32와 같은 exponent range	payload가 큼
FP8	8	E4M3 / E5M2	훈련·추론 가속	tensor별 scale 필요
INT4	4	균일한 integer grid	weight-only 생태계 성숙	outlier에 취약
FP4	4	E2M1 비균일 grid	작은 값 근처 상대 정밀도	16개 code뿐

양자화 = scale로 맞추고 grid에 반올림

$$q = \text{clip}(\text{round}(x / s) + z)$$

$$\hat{x} = s \cdot (q - z)$$

S

scale: 눈금 간격
값의 범위를 grid에 맞춤

Z

zero-point: 실수 0이
어떤 정수 code인지 지정

Symmetric vs. asymmetric

Symmetric은 보통 $z=0$. Asymmetric은 z 를 움직여 한쪽으로 치우친 분포를 더 잘 담습니다. NVFP4는 scale-only 대칭 표현입니다.

작은 예제로 보면 오차가 어디서 생기는지 보인다

교육용 대칭 INT4 예시 · 실제 NVFP4 변환식이 아님 · $q_{\max}=7$

단계	①	②	③	④
원래 x	-2.40	-0.70	0.20	1.90
$q = \text{round}(x/s)$	-7	-2	1	6
복원 $\hat{x} = q \cdot s$	-2.40	-0.69	0.34	2.06
오차	0.00	0.01	0.14	0.16

세 가지 손실

- ① rounding
- ② clipping/saturation
- ③ underflow-to-zero

비트가 줄수록 세 손실이 커집니다.

Scale 범위가 quantization 정확도를 좌우한다



Per-tensor

tensor 전체가
scale 하나 공유



Per-channel

행/출력 채널마다
scale 하나



Per-group

몇 개 원소 묶음마다
scale 하나



Per-block

작은 2D/1D block마다
local scale

더 세밀한 scale → outlier 영향은 줄지만 metadata·계산 overhead는 증가

W4A16·W8A8·W4A4 · 무엇을 낮췄나?

W4A16

Weights 4-bit
Activations 16-bit

메모리 절감 중심

W8A8

Weights 8-bit
Activations 8-bit

Hopper/범용 저정밀 GEMM

W4A4

Weights 4-bit
Activations 4-bit

Blackwell native NVFP4
GEMM

Activation까지 4비트여야 native W4A4 Tensor Core 경로의 잠재력을 온전히 쓸 수 있다.

INT4는 균일, FP4는 비균일 grid

정규화된 양수 구간의 representable levels

INT4



FP4 E2M1



FP4도 scale 없이는 실제 tensor 범위를 담을 수 없습니다.

4. Training Methodology · 단순 FP4의 실패



naive baseline: 모든 linear + 모든 operand 1×16 scaling + 모두 nearest rounding → 조기 발산

① 민감한 layer

좁은 FP4 range·정밀도
→ 일부 BF16

② outlier · ③ weight 불일치

scale이 큰 값에 끌림 +
 $Q_{fwd}(W) \neq [Q_{bwd}(W^T)]^T$

④ gradient rounding bias

작은 update가 계속 0
→ stochastic rounding

NVFP4는 FP4 값과 두 종류 scale의 조합이다

1 E2M1 · 실제 4-bit 값

16개 code

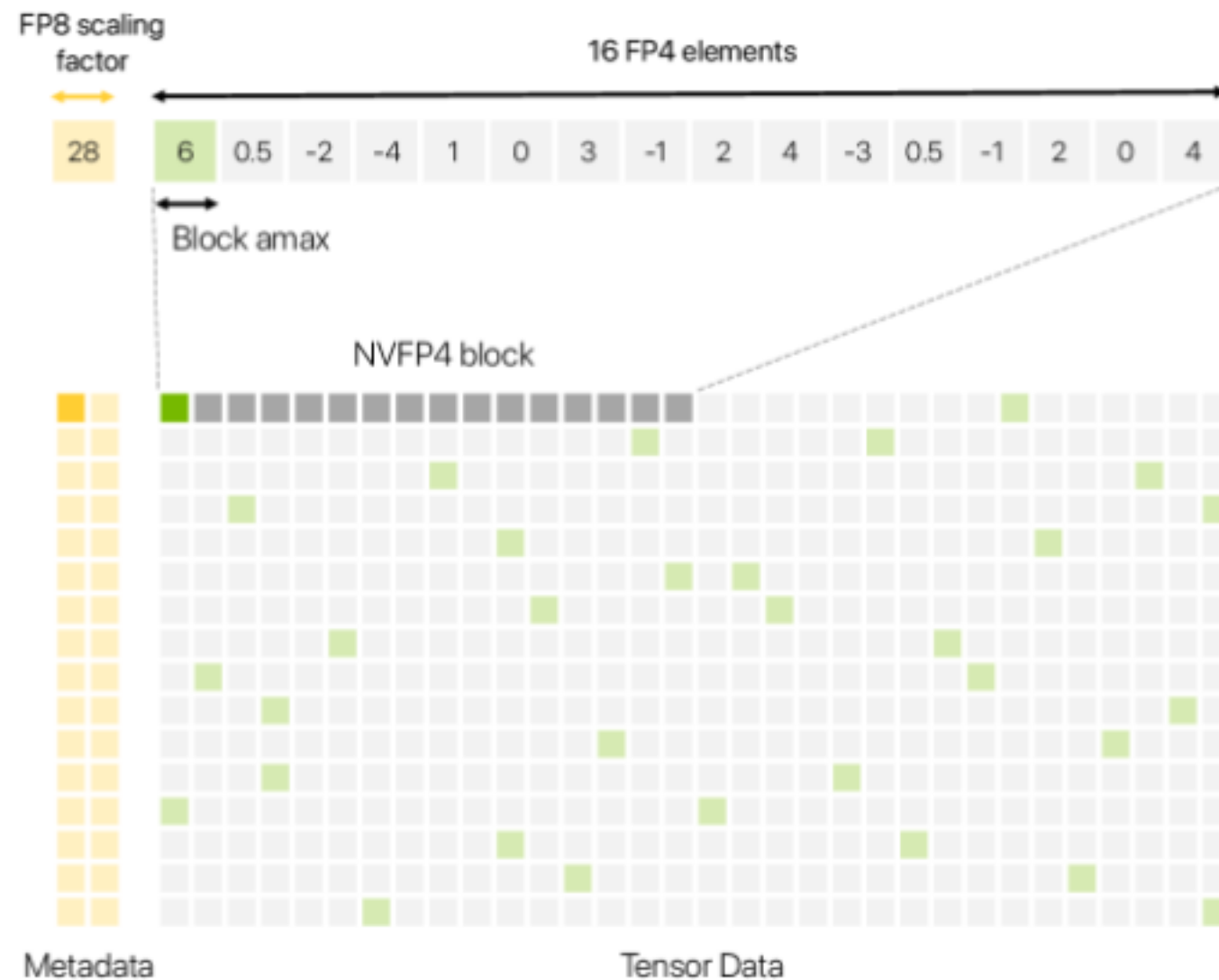
0, ± 0.5 , ± 1 , ± 1.5 , ± 2 , ± 3 , ± 4 , ± 6

2 Local · E4M3 block scale

내적 축의 연속 16개마다
지역 분포에 맞는 작은 자

3 Global · FP32 tensor scale

E4M3 block scale 전체가
표현 범위에 들어오도록 정규화



논문 사실 · $\text{FP4 value} \times \text{local E4M3 scale} \times \text{global FP32 scale}$

Local scale과 global scale은 역할이 다르다

Local · E4M3 · 16개마다

- 같은 block 안의 값 크기에 맞춤
- outlier 피해를 16개 안으로 제한
- FP4 code를 더 고르게 사용

Global · FP32 · tensor마다

- local scale들의 전체 범위를 맞춤
- 매우 크거나 작은 tensor를 정규화
- local scale을 없애는 방식이 아님

LOCAL × GLOBAL

두 scale을 함께 곱함

2-LEVEL

계층적 scaling · 2D scaling과 다름

MXFP4와 NVFP4 · 같은 E2M1, 다른 scale

MXFP4

32 values / block

E2M1 value + UE8M0 scale

- scale이 2의 거듭제곱
- 단순하지만 원하는 눈금과 차이가 날 수 있음
- block이 더 큼

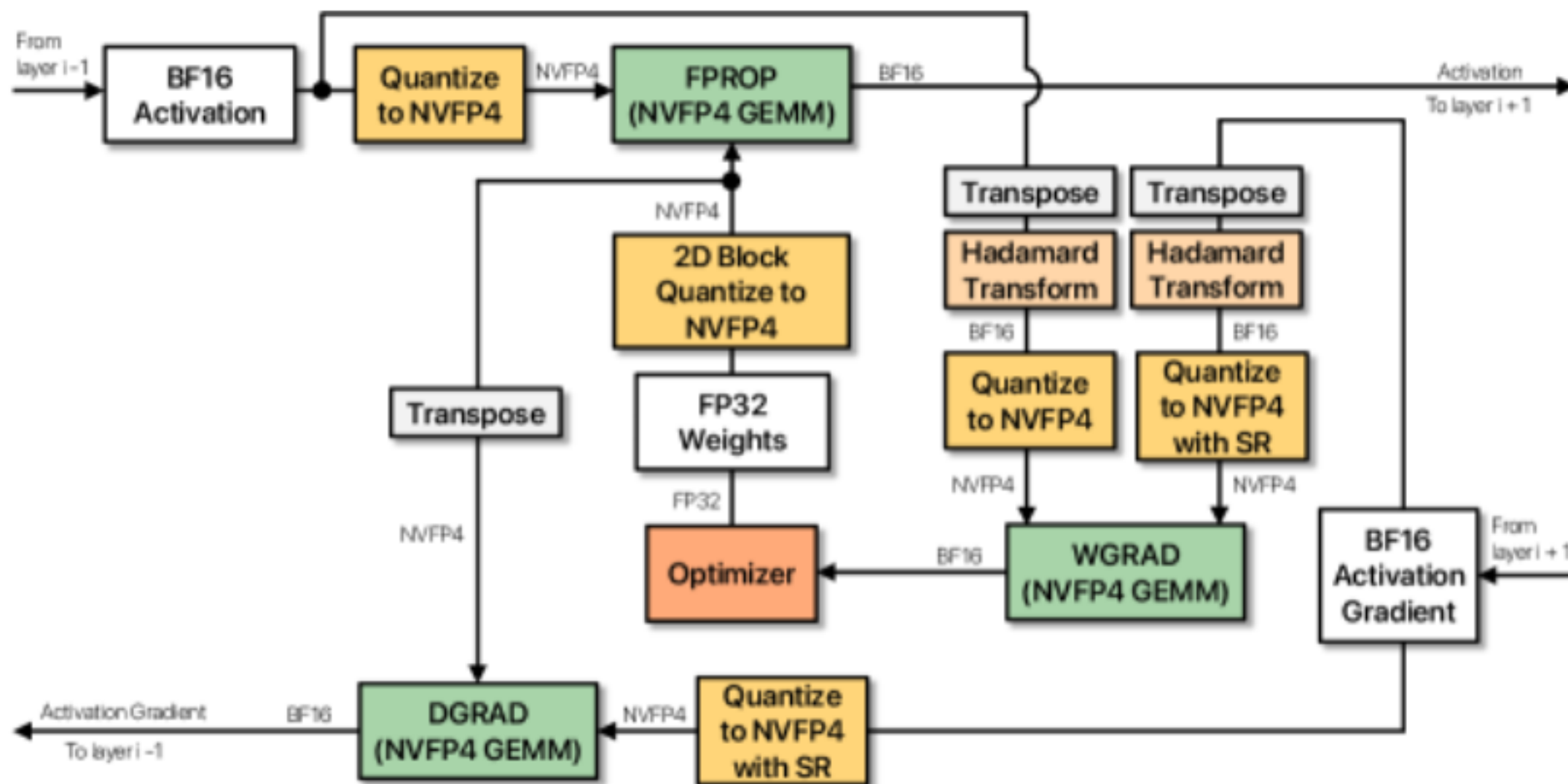
NVFP4

16 values / block

E2M1 value + E4M3
fractional scale

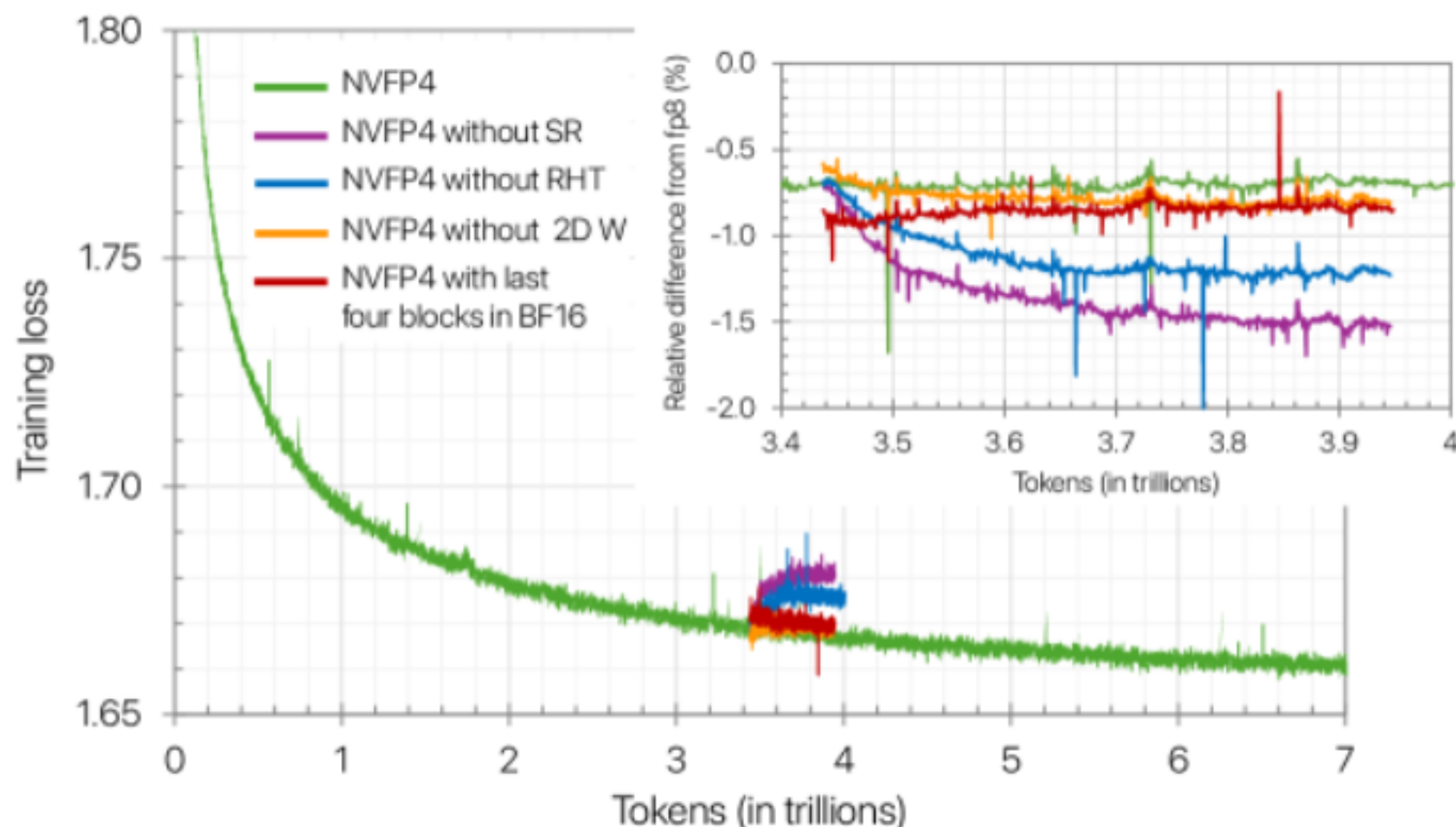
- 0.75 같은 fractional scale 가능
- 지역 분포를 더 촘촘히 맞춤
- tensor FP32 global scale 추가

NVFP4 training ≠ 전부 FP4



세 GEMM의 입력 operand만 NVFP4 · 출력/누산은 BF16/FP32 · scale metadata는 별도

Linear 하나에는 세 번의 큰 행렬곱이 있다



GEMM 입력은 NVFP4 · output/accumulation은 고정밀 · 왼쪽 Figure 4는 3.43T → 약 4T ablation 근거

① 민감층 → BF16

① FPROP · 예측

$$Y = XW \cdot \text{입력 } Q(X), Q(W)$$

② DGRAD · 앞 layer로

$$dX = dY W^T \cdot \text{입력 } Q(dY), Q(W^T)$$

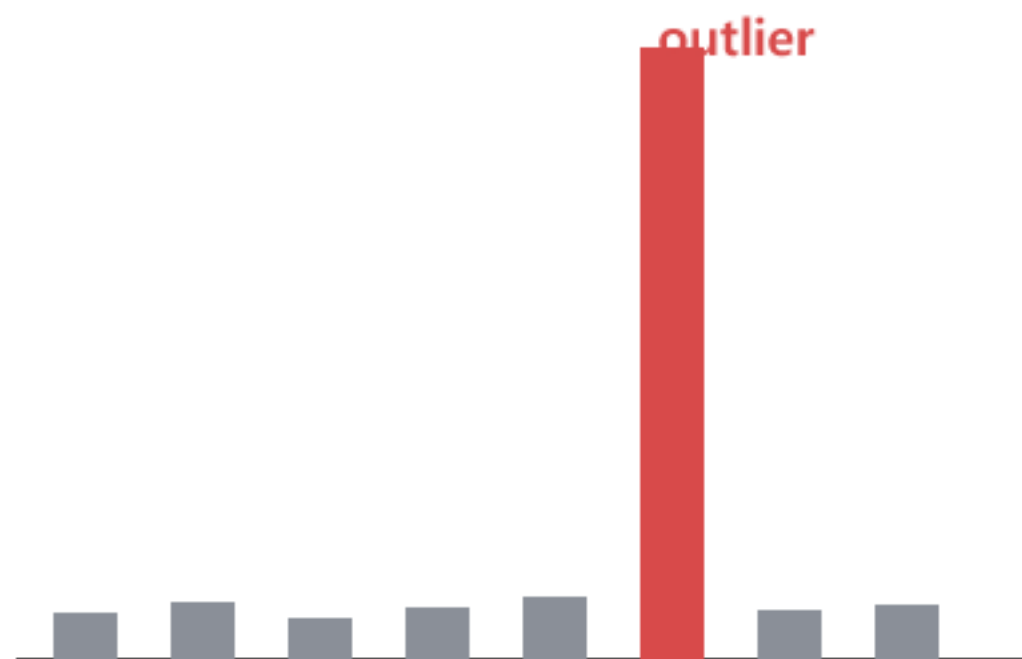
③ WGRAD · weight 변화량

$$dW = X^T dY \cdot \text{입력 } Q(X^T), Q(dY)$$

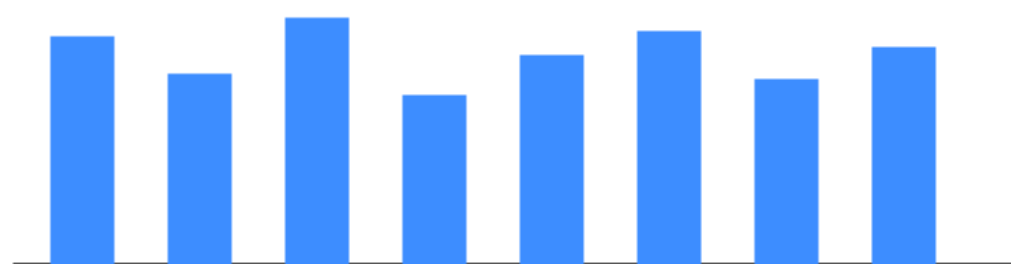
공통점

RHT는 outlier를 여러 좌표에 나눠 담는다

변환 전 · 큰 값 하나에 집중



RHT 후 · 16×16로 분산



논문 설정

WGRAD 두
입력

fixed sign
공유

$$(AH)(H^TB) = AB$$

2D scaling은 앞뒤에서 같은 W를 보게 한다

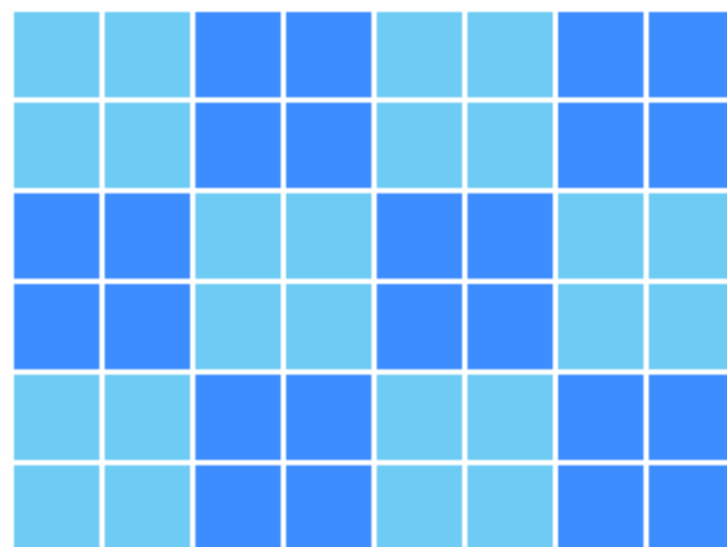
문제 · 방향별 1D quantization

$$Q_{\text{fwd}}(W) \neq [Q_{\text{bwd}}(W^T)]^T$$

Forward는 행, backward는 열 방향
→ 같은 W에서 다른 FP4 표현

해결 · weight-only 16×16 tile

정사각형 묶음은 transpose해도
동일

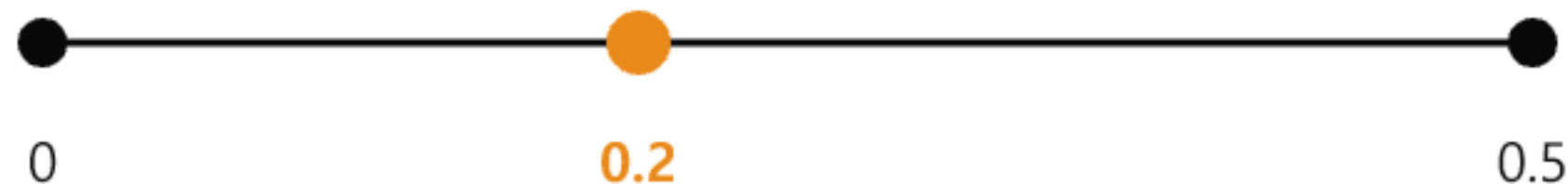


Activation·gradient는 1×16 scaling

2-level scaling ≠ 2D scaling · 앞은 scale 계층, 뒤는 weight tile 모양

Stochastic rounding은 작은 gradient를 살린다

설명용 예시 · 표현값 0과 0.5 · 실제 gradient $x=0.2$



RNE

0.2 → 항상 0
누적 bias

Stochastic

$P(0.5)=0.4$, $P(0)=0.6$
 $E[q]=0.2$

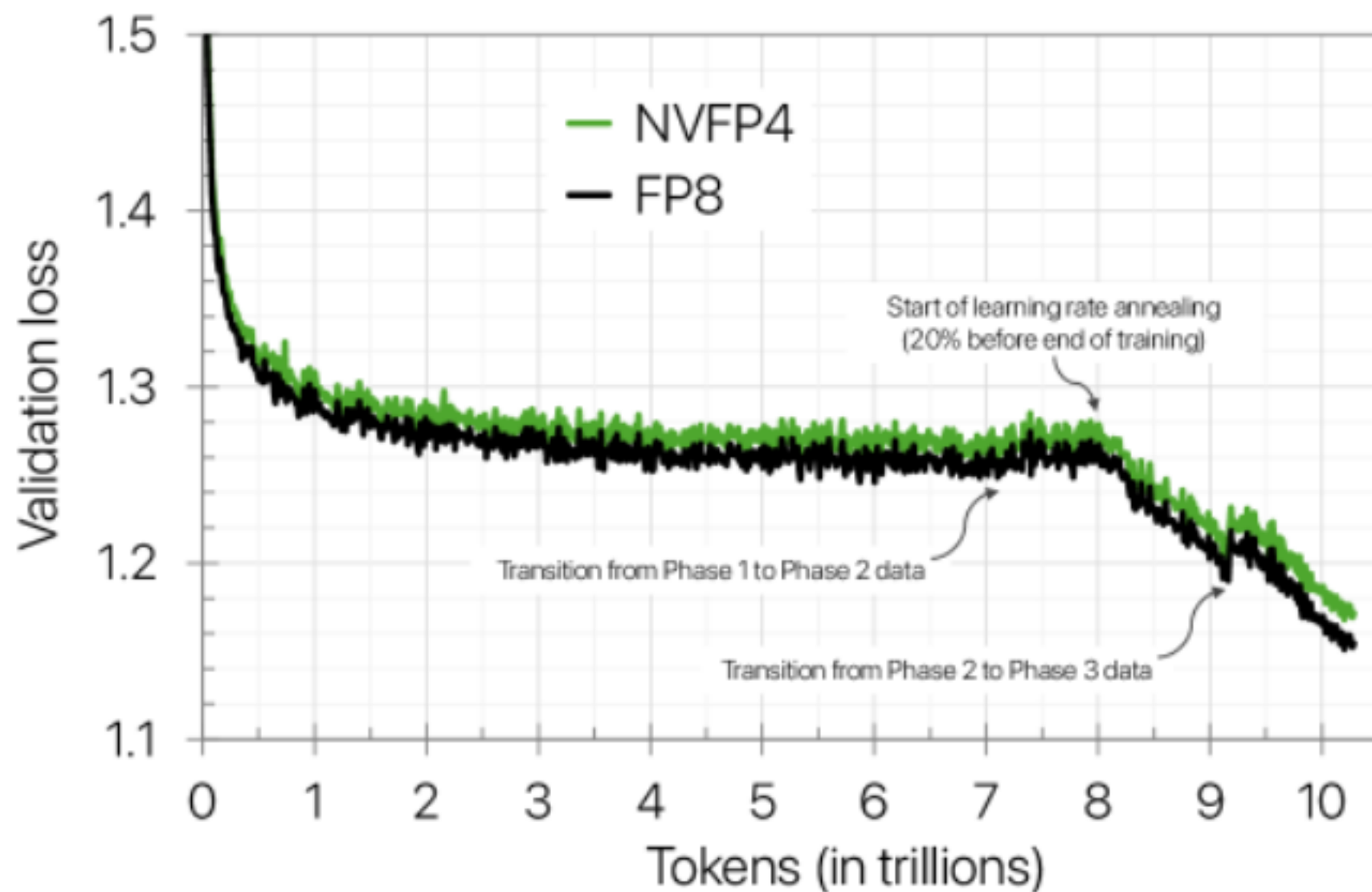
논문 recipe

Gradient: SR

Weight-Activation:
RNE

clipping 시 unbiased
보장 아님

Native NVFP4 · FP8과의 결과 비교



원 논문 Figure 2 · accuracy와 relative loss gap은 서로 다른 지표

62.58

NVFP4 MMLU-Pro accuracy (%)

FP8: 62.62

<1%

stable phase의
relative validation-loss gap vs FP8

정확한 해석

12B × 10T · 일부 linear와
비-GEMM은 고정밀
평가는 BF16 · Code는 FP8보다
낮음

Mixed precision은 역할별로 정밀도를 나눈다

NVFP4

큰 GEMM 입력을 낮춤

FPROP / DGRAD / WGRAD
W·A·gradient operand

BF16

output·민감 경로 보존

GEMM output·누산
attention·norm 등

FP32

학습 장부를 지킴

master W·accumulated dW
optimizer m, v

큰 행렬곱은 낮추고, 오랫동안 누적할 상태는 높게 유지한다.

네 장치가 서로 다른 고장을 함께 막는다

Pretraining Large Language Models with NVFP4 · Section 4

문제 → 해결

민감한 layer → selective BF16

outlier → WGRAD-only RHT

W / W^T 불일치 → weight-only 2D scaling

gradient bias → gradient-only stochastic

rounding

Ablation을 읽는 법

3.43T checkpoint에서 구성요소를 하나씩 제거해 약 4T까지 관찰했습니다. 하나의 마법이 아니라 조합의 필요성을 보인 근거입니다.

12B

hybrid model
10T training tokens

<1%

stable phase에서
FP8 대비 relative loss gap

Native NVFP4 결과 · 성공과 남은 한계

질문	논문이 보여준 것	해석의 경계	학생용 한 줄
수렴?	12B · 10T 학습 성공	모든 연산이 FP4는 아님	mixed precision으로 수렴
FP8과 같나?	stable phase loss gap <1%	후반 약 1.5%, Code gap 존재	가깝지만 동일하지 않음
속도?	저정밀 GEMM 가능성	wall-clock speedup 미보고	정확도 연구와 속도 실측 구분
정확도?	MMLU-Pro 62.58 vs FP8 62.62	Code 56.67 vs 59.52	task별 gap이 다름

전환 · 학습 수렴에서 정확도 복구로

PAPER A · NVFP4 Pretraining

시점: 모델을 처음부터 학습

문제: FP4 GEMM에서도 수렴할까?

해법: RHT · 2D scaling · SR · selective BF16

PAPER B · QAD

시점: BF16 post-training 완료 후

문제: NVFP4 inference 손실을 복구할까?

해법: frozen teacher의 분포를 forward KL로 모사

여기부터는 PAPER B · QAD — native training run을 이어가는 것이 아니라 완성 모델을 별도로 복구한다.

QAD의 질문 · NVFP4 정확도를 복구할까?

논문 문제 설정 · BF16 post-trained model → NVFP4 inference model

출발점 · 완성된 BF16

SFT

이미 완료

RL

이미 완료

원래 data·reward·pipeline을 다시 돌리지
않음

목표 · 행동 복구

PTQ

정확도 하락

QAD

BF16 분포에 접근

새 최고점을 만드는 training이 아니라 recovery

핵심: 같은 입력에서 frozen BF16 teacher가 내는 soft distribution을 student가 따라간다.

PTQ·QAT·QAD · 무엇을 어떻게 학습하나?

PTQ

weight gradient fine-tuning 없음
calibration으로 scale·clipping·rounding
등을 정함

오해 방지

일반 QAT가 hard label 전용인 것은 아닙니다. PTQ도 weight를 end-to-end 재학습하지 않을 뿐 quantization parameter는 최적화할 수 있습니다.

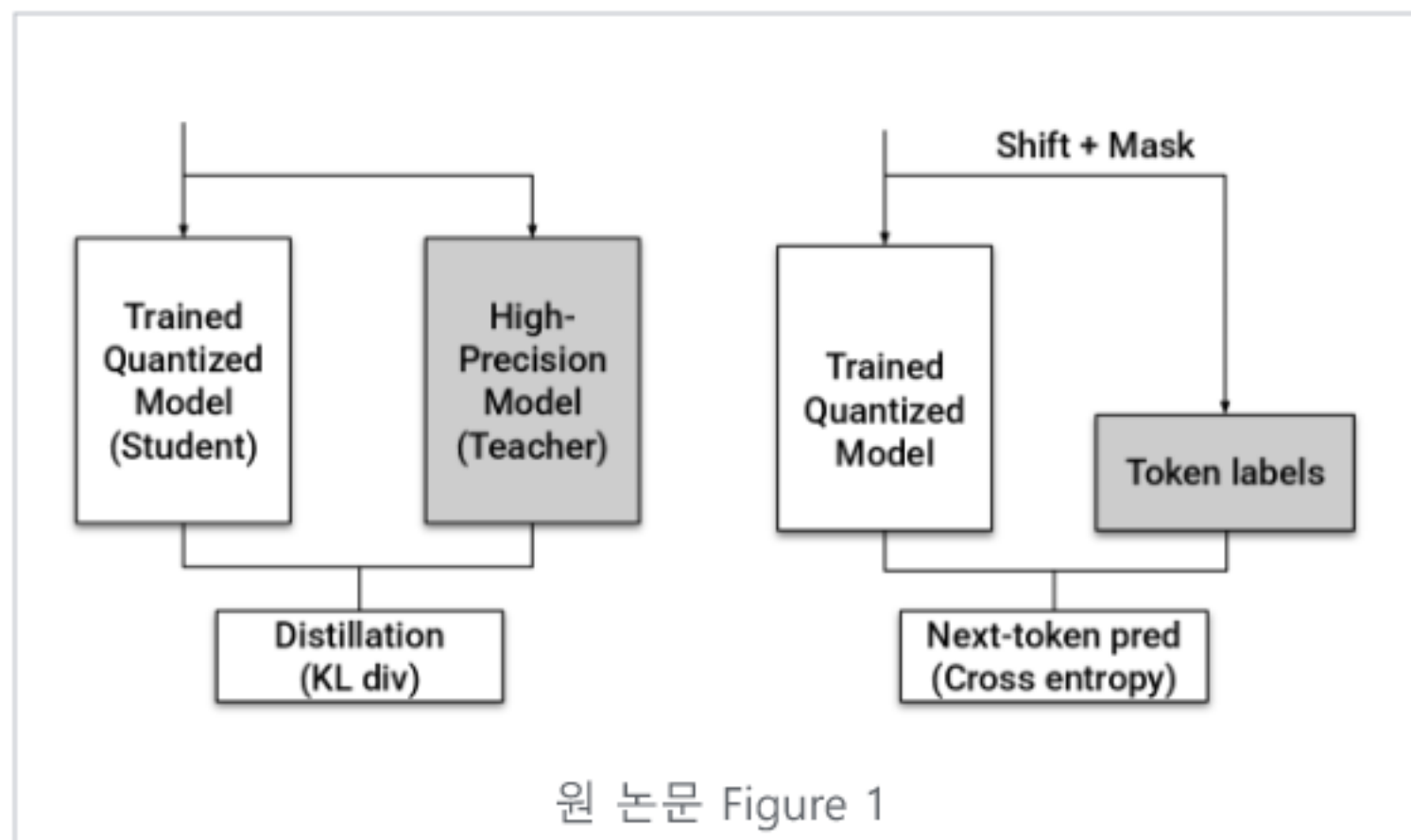
QAT

quantization-aware forward +
task loss
이 논문 baseline은 next-token
CE

QAD

QAT의 special case + teacher
distillation
forward KL로 원본 분포 복구

QAD는 새 능력보다 “양자화 전 행동”을 복구한다



Teacher

원본 BF16
frozen

Student

같은 시작점
simulated NVFP4

같은 모델이 왜 teacher가 될 수 있을까?

일반 distillation

큰 teacher → 작은 student

Teacher

원본 BF16
frozen

Student

같은 시작점
simulated NVFP4

QAD self-distillation

목표는 지식 확장이 아니라 양자화 전 자기 분포 복원입니다.
Nano 9B AIME25: original 9B teacher 71.5 > larger 12B teacher 69.8.

QAD는 scale만 맞추는 방법이 아니다

논문 정의 + NVIDIA ModelOpt 공식 구현 기준

대상	상태	학습 중 역할	오해 방지
BF16 teacher	frozen	target distribution 생성	teacher update 없음
Student master weights	trainable · high precision	KL gradient로 update	scale-only가 아님
Quantization scales	ModelOpt recipe: frozen	simulated NVFP4 범위 결정	논문 본문은 세부 미명시
Student forward	fake/simulated quant	deployment error를 노출	실제 FP4 속도 측정이 아님

핵심

Teacher는 고정, student의 고정밀 master weights가 바뀐다. 공식 ModelOpt recipe에서는 scale은 고정한다.

두 FP4 학습은 목적과 계산 경로가 다르다

Native NVFP4 pretraining

처음부터 저정밀 GEMM으로 학습

FPROP · DGRAD · WGRAD 입력을 NVFP4로 낮춤

목표: 수렴을 지키며 training compute 절감

Fake-quant QAD

완성된 BF16 모델을 후처리

simulated NVFP4 student와 frozen teacher의 KL을 최소화

목표: inference 행동 복구

공통점은 양자화 오차를 학습 중 노출한다는 것 · native recipe와 post-hoc distillation은 서로 다른 방법

Fake quantization · 고정밀 저장, 저정밀 계산

교육용 uniform grid · $\Delta=0.5$ · 실제 NVFP4는 E2M1 + two-level scale

단계	A	B	C	용도
Master w	1.60	0.60	-0.24	update할 고정밀 값
Quantize	1.50	0.50	0.00	FP4 오차 모사
Forward	1.50	0.50	0.00	student 계산에 사용
Backward	← gradient	← gradient	← gradient	master weights로 전달

세 줄 요약

Master weights는 연속값
Forward는 Q/DQ 값을 사용
Deployment 오차를 보며 학습

Backward는 quantizer를 어떻게 지나갈까?

Forward · 계단 함수

master weight는 고정밀로 저장

forward에서는 quantize→dequantize 값을 사용

round의 정확한 미분은 거의 0

Backward · STE

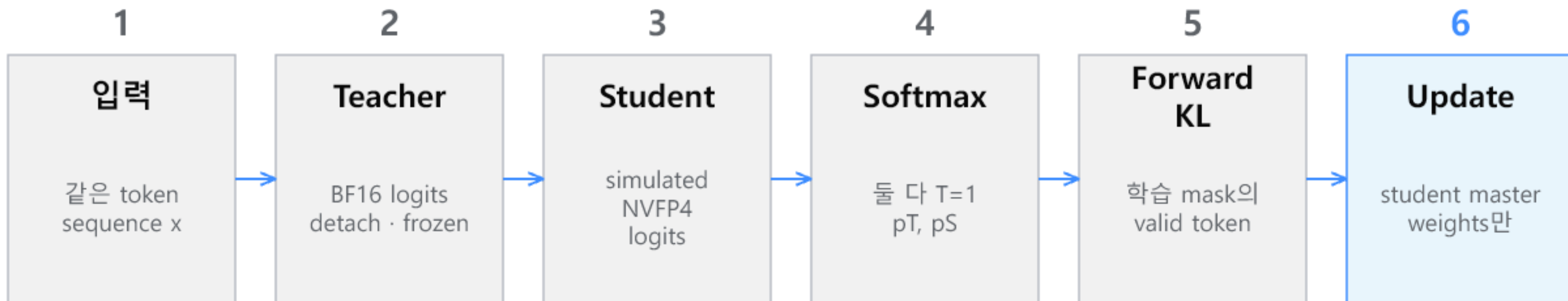
identity-like surrogate로 gradient를 통과시킴

student의 고정밀 master weight를 update

STE 세부는 구현 의존

범위: teacher는 frozen · student master weight는 update · ModelOpt 공식 recipe에서는 quantization scale 고정

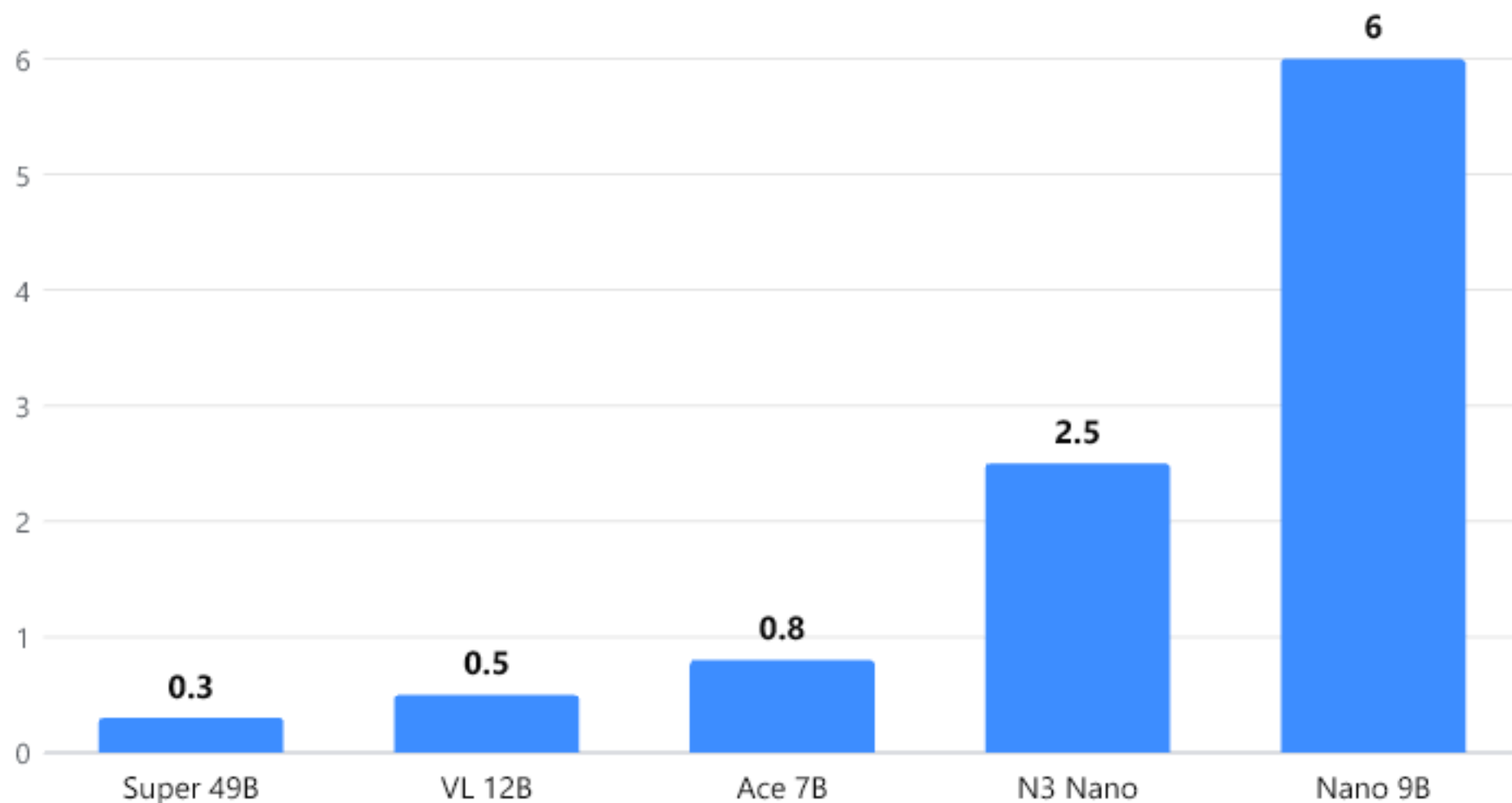
한 QAD batch · teacher 고정, student만 update



한 줄 의사코드

$pT = \text{softmax}(\text{teacher}(x), T=1).detach() \rightarrow pS = \text{softmax}(\text{student_FQ}(x), T=1) \rightarrow \text{KL}(pT \parallel pS) \rightarrow \text{student update}$

QAD training tokens · 모델마다 크게 다르다



축·단위 먼저

x: 모델 이름의 B = billion
parameters

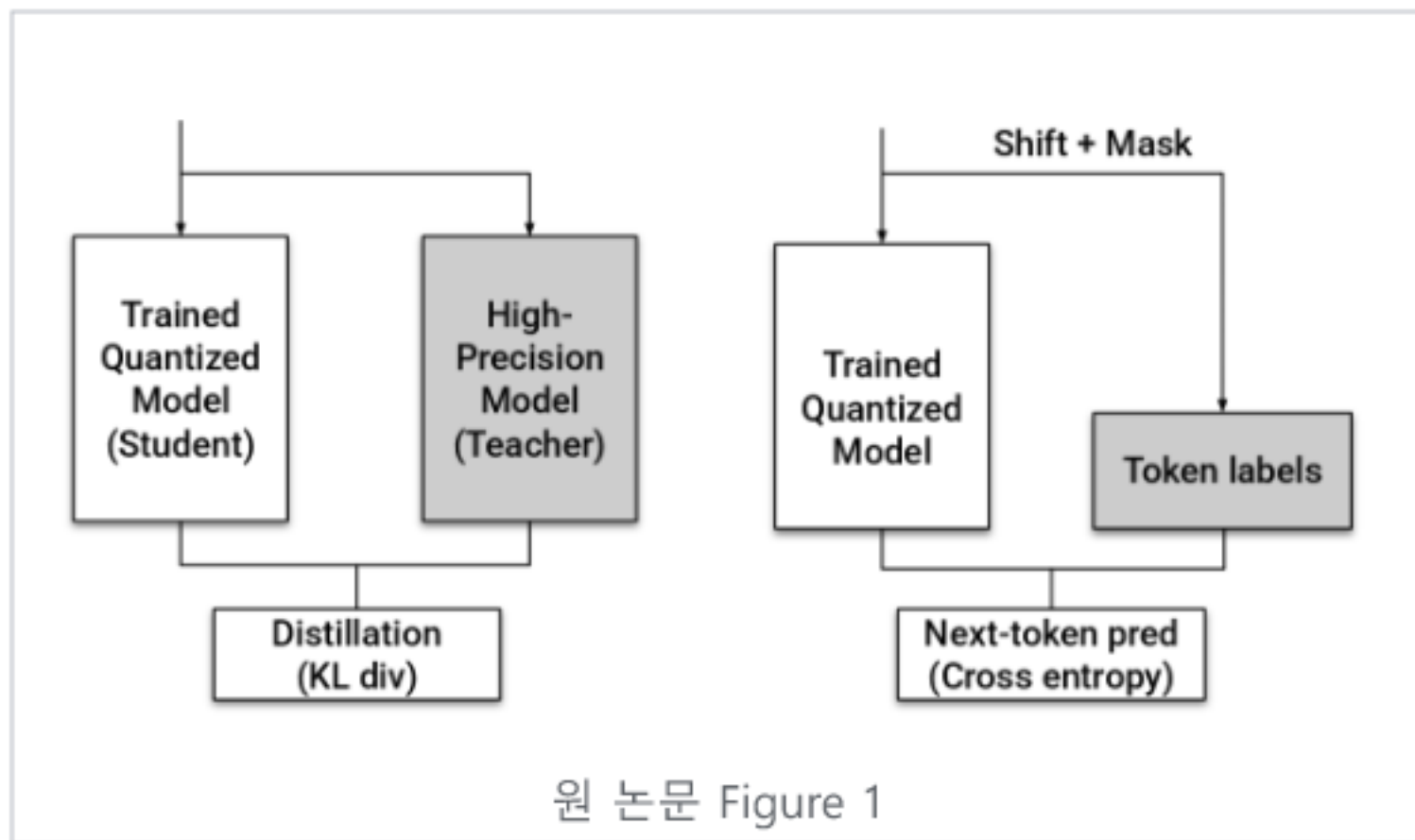
y: QAD tokens의 B = billion
tokens

비용 그래프가 아님

teacher forward + student
FWD/BWD 포함

GPU-hours·wall-clock 미보고
크기만으로 비용 추론 금지

Logit은 점수, softmax가 확률을 만든다



[설명용] $z = [2, 1, 0]$

$\text{softmax}(z) = [0.665, 0.245, 0.090]$

1등 A뿐 아니라 B가 C보다 더 가능하다는 정보가 남습니다.

왜 distribution을 쓰나?

정답 하나보다 후보 사이의 상대적 가능성까지 teacher가 전달하기 때문입니다.

Hard vs soft target · 같은 정답, 다른 정보량

$$z = [2, 1, 0] \cdot \text{logit}$$

$$\text{softmax}(z) = [0.665, 0.245, 0.090]$$

Hard

[1, 0, 0]
정답 token만

Soft

[0.665, .245, .090]
후보 관계까지

이 논문 baseline 한정

QAT baseline은 original task의 next-token CE, QAD는 frozen teacher의 distribution을 사용합니다. 일반 QAT가 hard label 전용이라는 뜻은 아닙니다.

KL은 비대칭 · QAD는 forward KL을 쓴다

Forward KL · 논문에서 사용

$D_{KL}(p_{\text{teacher}} \parallel p_{\text{student}})$

teacher가 중요하게 보는 token에 student가 충분한 확률을 주도록 맞춤

여기서 forward는 분포의 순서

Reverse KL · 미실험

$D_{KL}(p_{\text{student}} \parallel p_{\text{teacher}})$

순서를 바꾸면 gradient와 학습 성향이 달라짐

이 논문은 reverse KL을 비교하지 않음

검증된 결론: QAD objective는 $D_{KL}(p_T \parallel p_S)$ · reverse KL과의 우열은 미검증

T=1 · 검증된 recipe와 미검증 영역

학습 temperature T=1

teacher와 student 모두 기본 softmax
전체 vocabulary distribution이라 여전히
soft target

해석 범위

논문은 temperature alternatives를 ablation하지 않았습니다.
T=1은 distillation 학습용이며 평가 sampling T=0.6/1.0과 다른
개념입니다.

62.0

forward KL
Ace AIME25 accuracy

60.1

logit MSE
Ace AIME25 accuracy

QAD도 모델별 quantization 범위가 다르다

모델	후학습·구조	NVFP4 적용 범위	QAD tokens
Llama Super 49B	SFT-heavy · merge	모든 GEMM	0.3B
Nano 9B V2	hybrid	민감 attention·경계 BF16	6B
Nano 12B V2 VL	pretrain + 한 SFT stage	상세 범위 미보고	0.5B
N3 Nano 30B-A3B	RL-heavy MoE hybrid	일부 BF16 + KV FP8 + 나머지 NVFP4	2.5B
AceReason 7B	RL-heavy	모든 GEMM	0.8B
비교 원칙	같은 모델 안에서 PTQ/QAT/QAD 비교	모델 간 절대 점수 직접 비교 금지	B = billion tokens

Checkpoint 선택 · validation → benchmark



평가 설계에서 볼 점

보고할 downstream benchmarks가 checkpoint 선택에도 사용되어 optimistic bias 가능 · training seed/CI 미보고

CE가 같아도 원래 행동분포는 다를 수 있다

Llama Nemotron Super V1 · held-out 5,000 samples · 낮을수록 좋음

Cross Entropy vs labels

0.408

BF16

0.408

QAT

QAD의 CE는 0.416 · BF16과 같지 않음

KL divergence vs BF16

0.311

QAT

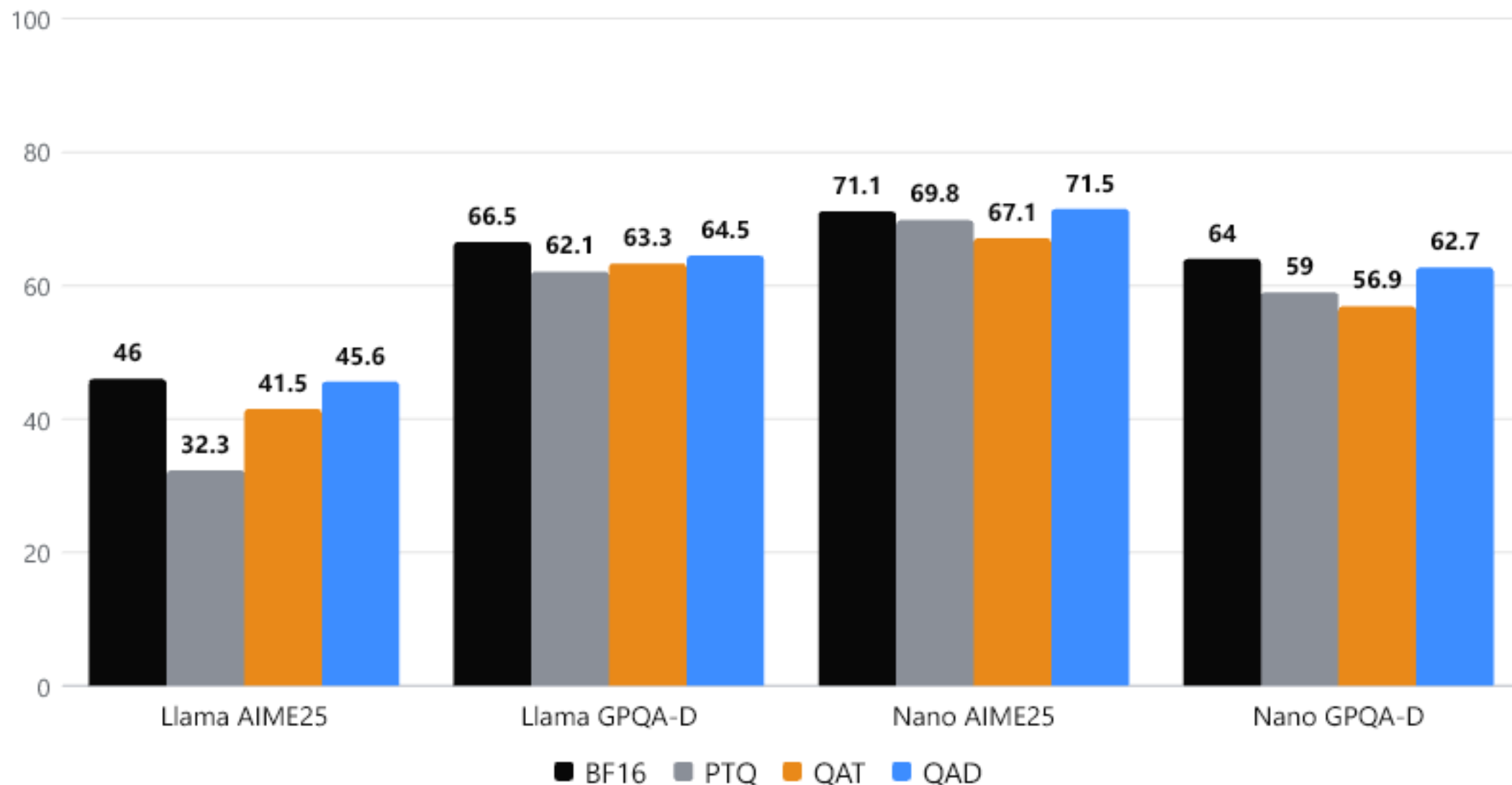
0.004

QAD

QAD가 맞춘 핵심은 teacher KL

Table 1: BF16 KL 0 / CE .408 · QAT .311 / .408 · QAD .004 / .416

SFT-heavy · QAD가 reasoning gap을 줄였다



Llama AIME25

accuracy ↑

BF16 46.0 · PTQ 32.3

QAT 41.5 · QAD 45.6

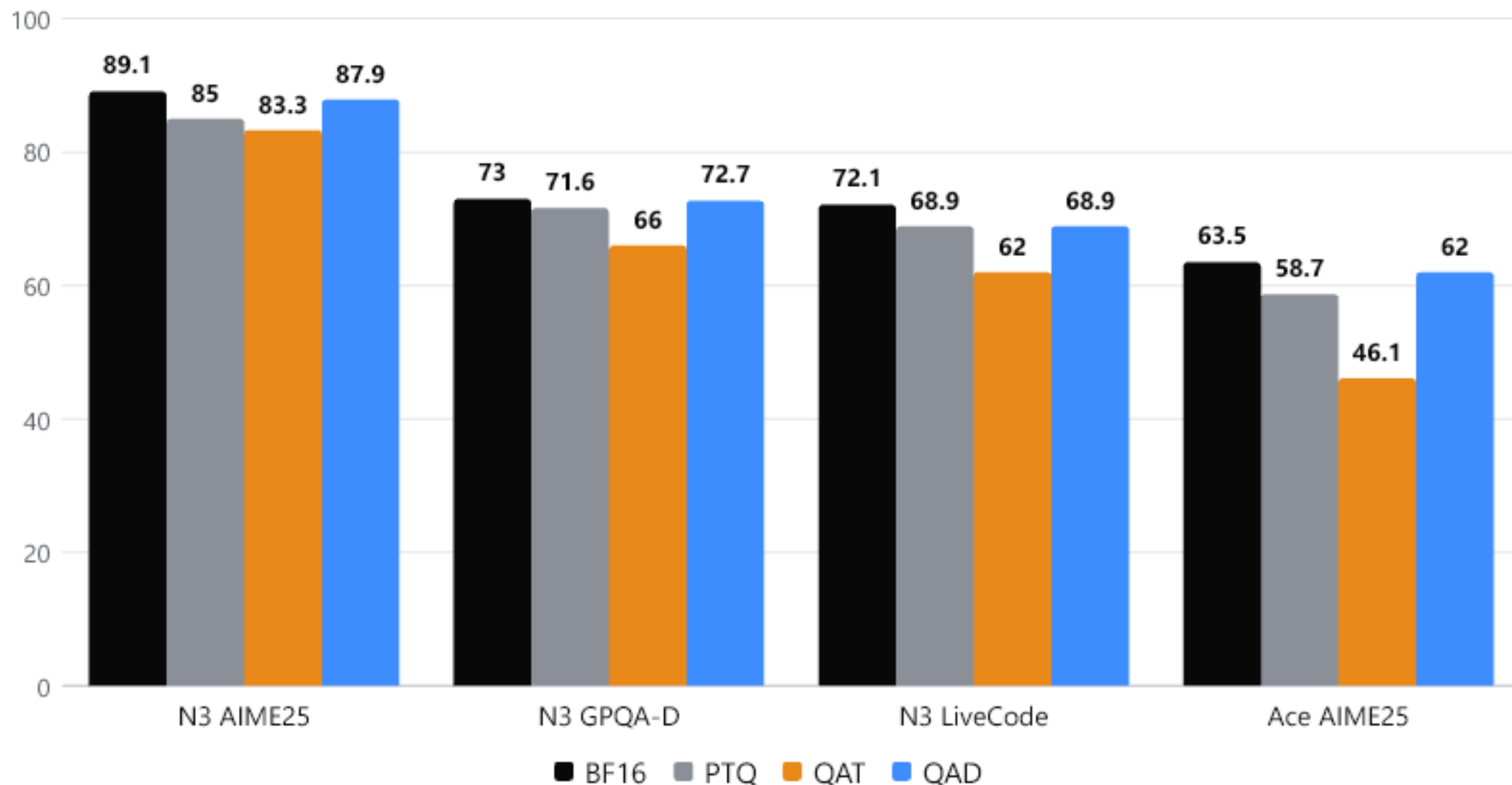
Reasoning gap 정의

BF16 benchmark score –
quantized score

QAD gap: 0.4%p

PTQ gap: 13.7%p

RL-heavy · QAT는 덮고 QAD는 보존했다



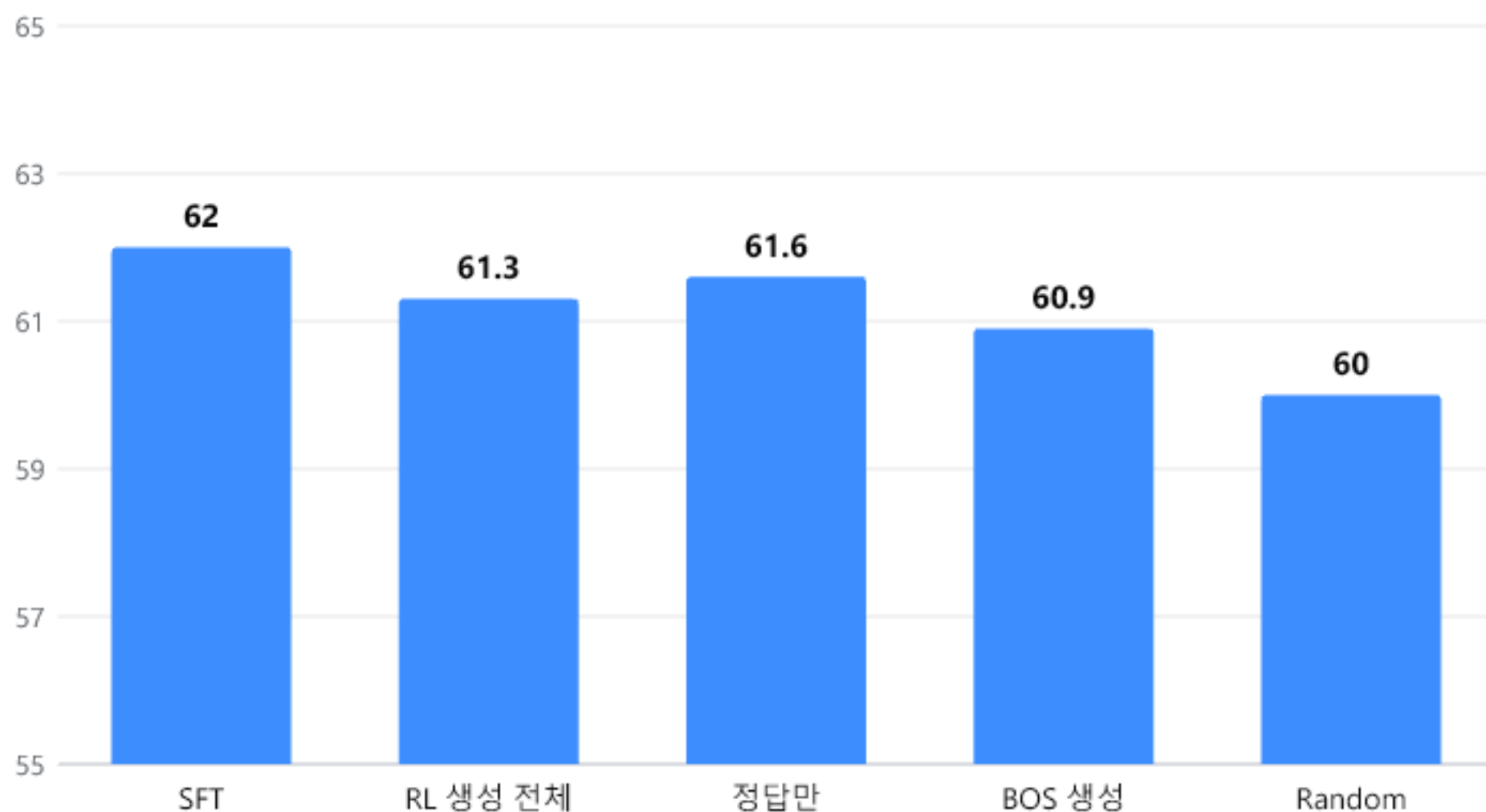
AceReason AIME25

accuracy ↑
 BF16 63.5 · PTQ 58.7
 QAT 46.1 · QAD 62.0

완전 복원은 아님

N3 LiveCode
 BF16 72.1
 PTQ 68.9 = QAD 68.9

불완전한 QAD data도 비교적 안정적이었다



AIME25 · y축 55–65%

Math-only 61.7

Code-only 62.0

Full mixture 62.0

Random-token 결과

AIME25 60.0 > PTQ 58.7

다른 지표는 PTQ보다 낮기도 함
"대체로 PTQ 수준, 망가뜨리지
않음"

QAD가 유리한 조건 · 한 모델에서는 $QAT \approx QAD$

QAD가 유리했던 사례

PTQ 정확도 손실이 큼

RL-heavy · multi-stage post-training

원래 행동 보존이 제한된 task 재학습보다 중요

$QAT \approx QAD$ 였던 사례

Nano 12B V2 VL · 한 모델

pretraining 뒤 하나의 SFT stage

저자는 simple pipeline과 작은 PTQ drop 때문이라고 추정

판단 순서: PTQ 평가 → 허용 손실 초과 시 QAD → 비용 대비 복원 폭 검증

비판적 검토 · 강점과 한계

강점

- multi-stage/RL 모델의 실제 문제 설정
- CE vs KL 진단이 명확
- 여러 model size·data mixture 비교
- 간단한 objective와 conservative recipe

한계

- NVIDIA/Nemotron·NVFP4 중심
- GPU-hours·wall-clock·teacher memory 미보고
- training seeds·confidence intervals 없음
- benchmark를 checkpoint 선택에도 사용

Near-BF16은 완전 복원 보장이 아니다 · label-free는 input-free가 아니다 · INT4/MXFP4 일반화는 추가 검증 필요

예상 질문 1-2 · QAD의 정체

Q1. QAD와 QAT는 무엇이 다른가?

이 논문 QAT baseline: original task의 next-token CE

QAD: frozen BF16 teacher의 전체 분포를 forward KL로 복구

QAD는 distillation loss를 쓰는 QAT의 special case

Q2. 같은 모델도 teacher인가?

가능합니다. 목표는 크기 축소가 아니라 양자화 전 자기 행동 복원입니다.

Nano 9B AIME25
original teacher 71.5 > larger 12B teacher 69.8

한 줄 답: “더 강한 교사”보다 “원래 분포와 같은 교사”가 복원 기준에 맞다.

Nemotron 3 Super · PTQ + AutoQuantize

실제 경로

Native NVFP4 pretraining

- BF16 base/master checkpoint
- BF16 SFT / RL post-training
- ModelOpt PTQ + AutoQuantize
- mixed deployment checkpoint

AutoQuantize가 한 일

- 후보: NVFP4 / FP8 / BF16
- operator별 sensitivity + cost search
- effective budget ≈ 4.75 bits
- weight update-QAD 아님
- router/cache/BMM은 별도 고정 precision

<2h · 8×B200 · 512 samples · seq 4096 · BF16 대비 median relative accuracy 99.8% (모든 benchmark 보장 아님)

예상 질문 3-4 · 데이터와 업데이트 대상

Q3. Label-free인가, data-free인가?

정답 label은 필요 없지만 입력 sequence는 필요합니다. teacher가 학습 mask의 valid token에 soft target을 만듭니다.

정확한 표현: label-free / original-data-free

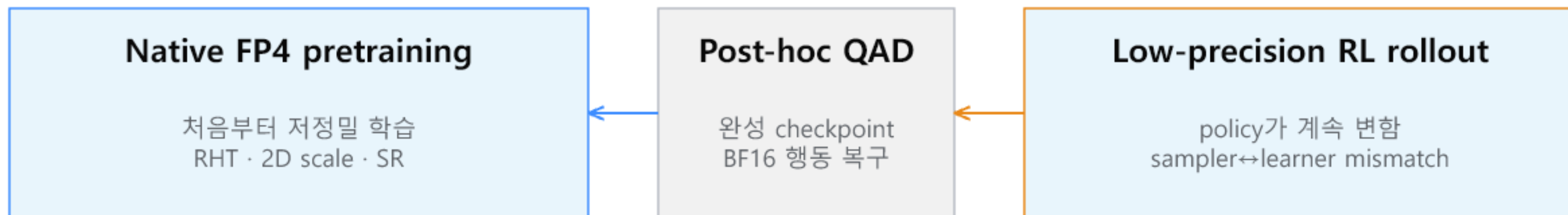
Q4. QAD는 scale만 학습하나?

아닙니다. student의 고정밀 master weights가 KL gradient로 바뀝니다.

공식 ModelOpt recipe에서는 scale은 고정하고 weights를 update합니다.

논문 정의와 구현을 분리: “quantized student 최적화”는 논문, “frozen scales”는 ModelOpt 구현.

세 연구 축은 이어지지만 같은 문제는 아니다



축	대표 질문
Speculative decoding	target verifier distribution을 보존하며 generation을 빠르게?
QAD	RL 종료 뒤 quantized checkpoint의 accuracy를 복구?
QARL / QaRL	저정밀 rollout과 learner를 alignment?
QUADS	NVFP4 W4A4 MoE RL의 activation error를 어떻게 막나?

병렬

가속 축
인과 순서 아님

QAD

RL 중 mismatch를
직접 해결하지 않음

예상 질문 5-6 · KL 방향과 temperature

Q5. 왜 $KL(\text{teacher} \parallel \text{student})$ 인가?

Forward KL은 teacher가 중요하게 보는 token을 기준으로 student를 맞춥니다. KL은 비대칭이라 순서를 바꾸면 다른 objective입니다.

논문은 reverse KL을 비교하지 않았습니다.

Q6. 왜 temperature $T=1$ 인가?

원본 teacher distribution을 그대로 복구하는 recipe입니다. 전체 vocabulary 분포라 $T=1$ 이어도 soft target입니다.

다른 T 와의 비교는 미보고입니다.

$T=1$ 은 distillation 학습 softmax temperature · 평가 sampling temperature와 다른 개념

예상 질문 7-8 · 두 FP4 학습과 FP32 장부

Q7. Native와 QAD는 같은가?

아닙니다. Native는 처음부터 저정밀 FPROP·DGRAD·WGRAD GEMM으로 학습합니다. QAD는 완성 모델을 simulated NVFP4 forward와 KL로 복구합니다.

Q8. 왜 optimizer는 FP32인가?

작은 update, accumulated dW, Adam m/v를 오래 누적해야 하기 때문입니다. FP4의 이득은 큰 GEMM operand를 낮추는 데서 옵니다.

한 줄 답: “계산용 저정밀”과 “학습 장부용 고정밀”을 역할별로 섞는 것이 mixed precision이다.

예상 질문 9-10 · 완전 복원과 RL 확장

마지막 오해 두 개 · QAD는 강력하지만 만능은 아니다

Q9. 항상 BF16까지 완전히 복원하나?

72.1

N3 LiveCode ·
BF16

68.9

PTQ = QAD

Near-BF16은 보장 문구가 아님 · task별
gap이 남음

Q10. RL rollout mismatch도 직접 해결하나?

QAD

RL 종료 후
checkpoint 복구

QARL

RL 중
sampler↔learner
alignment

SpecDec·QAD·QARL은 서로 다른 축

최종 한 문장 · FP4의 승부는 “4bit를 켜는 것”이 아니라, 어디를 낮추고 어떤 오차를 어떤 recipe로 막는가에서 난다.